TREE

1. DFS Traversal Pattern

Learn recursion and tree traversal.

Questions

- Binary Tree Inorder Traversal

```
public:
void inorder(TreeNode* root,vector<int>& ans){
    if(root==nullptr) return ;
    inorder(root->left,ans);
    ans.push_back(root->val);
    inorder(root->right,ans);
}
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int>ans;
        inorder(root,ans);
        return ans;
    }
};
```

- Binary Tree Preorder Traversal

```
void inorder(TreeNode* root,vector<int>& ans){
    if(root==nullptr) return ;
        ans.push_back(root->val);
    inorder(root->left,ans);
      inorder(root->right,ans);
}
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int>ans;
        inorder(root,ans);
        return ans;

    }
```

- Binary Tree Postorder Traversal

```
void inorder(TreeNode* root,vector<int>& ans){
    if(root==nullptr) return ;
    inorder(root->left,ans);
    inorder(root->right,ans);
        ans.push_back(root->val);

}
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int>ans;
        inorder(root,ans);
        return ans;
```

Pattern

```
dfs(root->left);
dfs(root->right);
```

---

2. BFS / Level Order Pattern ★★★

Queue-based tree problems.

Questions

- Binary Tree Level Order Traversal

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>>ans;
    queue<TreeNode*>q;
    if(root==nullptr) return ans;
    q.push(root);
    while(!q.empty()){
     int n=q.size();
     vector<int>level;
     for(int i=0;i<n;i++){
        TreeNode* node=q.front();
        q.pop();
        level.push_back(node->val);
        if(node->left){
            q.push(node->left);
        }
         if(node->right){
            q.push(node->right);
        }
     }
  ans.push_back(level);

    }
return ans;
```

- Binary Tree Zigzag Level Order Traversal

```
vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
vector<vector<int>>ans;
queue<TreeNode*>q;
if(root==nullptr) return ans;
q.push(root);
while(!q.empty()){
    vector<int>level;
    int n=q.size();
    for(int i=0;i<n;i++){
        TreeNode* node=q.front();
        q.pop();
        level.push_back(node->val);
        if(node->left){
            q.push(node->left);
        }
        if(node->right){
            q.push(node->right);
        }
    }
    if(ans.size()%2==1){
        reverse(level.begin(),level.end());
    }
    ans.push_back(level);
}
   return ans;
}
```

- Binary Tree Right Side View

```
    vector<int>ans;
    if(root==nullptr) return ans;
    queue<TreeNode*>q;
    q.push(root);
    while(!q.empty()){
        int n=q.size();
        vector<int>level;
        for(int i=0;i<n;i++){
            TreeNode* node=q.front();
            q.pop();
            level.push_back(node->val);
            if(node->left){
                q.push(node->left);
            }
            if(node->right){
                q.push(node->right);
            }
        }
        ans.push_back(level.back());

    }
    return ans;

}
```

- Average of Levels in Binary Tree

```cpp
vector<double> averageOfLevels(TreeNode* root) {
        vector<double>ans;
    if(root==nullptr) return ans;
    queue<TreeNode*>q;
    q.push(root);
    while(!q.empty()){
        int n=q.size();
        vector<int>level;
        double sum=0;
        for(int i=0;i<n;i++){
            TreeNode* node=q.front();
            q.pop();
            sum+=node->val;
            if(node->left){
                q.push(node->left);
            }
            if(node->right){
                q.push(node->right);
            }
        }
        ans.push_back(sum/n);
    }
    return ans;
```

Pattern

`queue<TreeNode*> q;`

Process nodes level by level.

---

3. Height / Bottom-Up DFS Pattern ★★★

Return information from children to parent.

Questions

- Maximum Depth of Binary Tree -

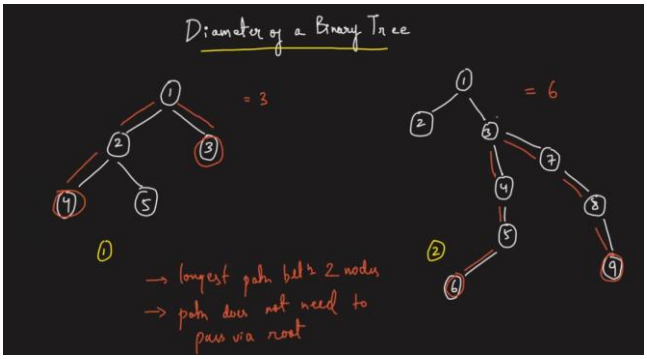
```cpp
int maxDepth(TreeNode* root) {
    if(root==nullptr) return 0;
    return 1+max(maxDepth(root->left),maxDepth(root->right));
```

- Balanced Binary Tree –

```cpp
int balanced(TreeNode* root){
    if(root==nullptr) return 0;

       int left=balanced(root->left);
       if(left==-1) return -1;

       int right=balanced(root->right);
       if(right==-1) return -1;


       if(abs(left-right)>1) return -1;
       return 1+max(left,right);
}
    bool isBalanced(TreeNode* root) {
      int x= balanced(root);
       return x!=-1;

    }
```

- Diameter of Binary Tree – f

  inal answer toh **diameter hi nikalna hai**, lekin diameter nikalne ke liye har node par **height ki zaroorat padti hai**.




```
int check(TreeNode* root,int &dia){
    if(root==nullptr) return 0;
    int left=check(root->left,dia);
    int right=check(root->right,dia);
    dia=max(dia,left+right);
    return 1+max(left,right);
}
    int diameterOfBinaryTree(TreeNode* root) {
            int dia=0;
        check(root,dia);
        return dia;

    }
```

Pattern

```
return 1 + max(left,right);
```

---

4. Path Sum Pattern ★★★

Root-to-leaf and any-path problems.

Questions

- Path Sum

```
bool hasPathSum(TreeNode* root, int targetSum) {
    if(root==nullptr) return 0;
if(root->left==nullptr&&root->right==nullptr){
    if(targetSum==root->val){
        return true;
    }
}
    return hasPathSum(root->left,targetSum-root->val)||hasPathSum(root->right,targetSum-root->val);
```

- Path Sum II

```cpp
void solve(TreeNode* root,int target,vector<int>& path,vector<vector<int>>& ans){
    if(root==nullptr) return;
    path.push_back(root->val);
    if(root->left==nullptr&& root->right==nullptr){
    if(target==root->val){
        ans.push_back(path);
    }
    }
    solve(root->left,target-root->val,path,ans);
    solve(root->right,target-root->val,path,ans);
        path.pop_back();
}
    vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
        vector<int>path;
        vector<vector<int>>ans;
        solve(root,targetSum,path,ans);
        return ans;
```

- Path Sum III
- Binary Tree Maximum Path Sum –almost same as diameter of binary tree

```cpp
class Solution {
public:
int  check(TreeNode* root,int& dia){
    if(root==nullptr) return 0;

    int left=max(0,check(root->left,dia));
    int right=max(0,check(root->right,dia));
    dia=max(dia,left+right+root->val);
    return root->val+max(left,right);
}

    int maxPathSum(TreeNode* root) {
        int dia=INT_MIN;
        check(root,dia);
        return dia;
    }
};
```

Pattern

Pass current sum while traversing.

---

5. Tree Construction Pattern ★★★

Build tree from traversals.

Questions

- Construct Binary Tree from Preorder and Inorder Traversal
- Construct Binary Tree from Inorder and Postorder Traversal

Pattern

Find root in inorder and divide.

---

6. LCA Pattern ★★★

One of the most important interview patterns.

Questions

- Lowest Common Ancestor of a Binary Tree
- Lowest Common Ancestor of a BST

Pattern

```
if(root==p || root==q)
    return root;
```

---

7. BST Property Pattern ★★★

Use BST ordering.

Questions

- Search in a BST

```
TreeNode* searchBST(TreeNode* root, int val) {
    if(root==nullptr) return nullptr;

    if(root->val==val){
        return root;
    }
    else if(root->val>val){
        return searchBST(root->left,val);
    }
    else{
        return searchBST(root->right,val);
    }
    return 0;
```

- Insert into a BST

```
TreeNode* insertIntoBST(TreeNode* root, int val) {
    if(root==nullptr){
        return new TreeNode(val);//Jab recursion kisi empty position tak pahunch jati hai, wahan naya node bana dete hain.
    }
    if(root->val>val){
        root->left=insertIntoBST(root->left,val);
    }
    else if(root->val<val){
        root->right=insertIntoBST(root->right,val);
    }
    return root;
```

- Validate BST

```
public:
void inorder(TreeNode* root,vector<int>& ans){
    if(root==nullptr) return ;
    inorder(root->left,ans);
    ans.push_back(root->val);
    inorder(root->right,ans);
}
    bool isValidBST(TreeNode* root) {
     vector<int>ans;
     inorder(root,ans);
     for(int i=1;i<ans.size();i++){
        if(ans[i]<ans[i-1]){
            return false;
        }
     }
     return true;
    }
};
```

- Kth Smallest Element in BST

```
void inorder(TreeNode* root,vector<int>& ans){
    if(root==nullptr) return ;
    inorder(root->left,ans);
    ans.push_back(root->val);
    inorder(root->right,ans);

}
    int kthSmallest(TreeNode* root, int k) {
    vector<int>ans;
    inorder(root,ans);
    return ans[k-1];
    }
};
```

- Delete Node in a BST

Pattern

```
left < root < right
```

---

8. Tree Comparison Pattern

Compare two trees.

Questions

- Same Tree

```
*/
if(p==nullptr&&q==nullptr) return true;
if(p==nullptr||q==nullptr) return false;
if(p->val!=q->val) return false;
return isSameTree(p->left,q->left)&&isSameTree(p->right,q->right);
```

- Symmetric Tree

```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if(root==nullptr) return false;
        return ismirror(root->left,root->right);

    }
    bool ismirror(TreeNode* left,TreeNode* right){
        if(left==nullptr&& right==nullptr) return true;
        else if(left==nullptr&&right!=nullptr) return false;
        else if(left!=nullptr&&right==nullptr) return false;
        return (left->val==right->val)&&ismirror(left->left,right->right)&&ismirror(left->right,right->left);
    }
```

- Subtree of Another Tree

---

9. Backtracking on Trees Pattern

Maintain path and backtrack.

Questions

- Binary Tree Paths
- Path Sum II

Pattern

```
path.push_back(root->val);
...
path.pop_back();
```

---

10. View / Vertical Traversal Pattern ⋆ ⋆ ⋆

Very common in interviews.

Questions

- Binary Tree Right Side View -done
- Vertical Order Traversal of a Binary Tree
- Top View (GFG)
- Bottom View (GFG)

Pattern

Use:

```
map<int, vector<int>>
```

Store nodes by column.

---

11. Serialization Pattern

Questions

- Serialize and Deserialize Binary Tree

Pattern

Store null nodes while traversal.

---

12. Tree DP Pattern (Advanced) ★ ★ ★

DP on trees.

Questions

- House Robber III
- Binary Tree Cameras

Most Important Order (Recommended)

For your preparation:

Week 1-done

1. Inorder Traversal -doone
2. Level Order Traversal -done
3. Maximum Depth -done
4. Balanced Binary Tree -done
5. Diameter of Binary Tree -done

Week 2

6. Path Sum -done
7. Path Sum II -done
8. LCA of Binary Tree
9. Same Tree -done
10. Symmetric Tree -done

Week 3

11. Validate BST -done
12. Kth Smallest in BST -done
13. Construct Tree from Traversals
14. Right Side View -done
15. Vertical Order Traversal

Week 4

16. Binary Tree Maximum Path Sum
17. Serialize & Deserialize
18. House Robber III

Minimum Absolute Difference in BST

```
int pre=-1;
void inorder(TreeNode* root,int &mine){
    if(root==nullptr) return;
    inorder(root->left,mine);
    if(pre!=-1){
    mine=min(mine,root->val-pre);
    }
    pre=root->val;
    inorder(root->right,mine);
}
    int getMinimumDifference(TreeNode* root) {
        int mine=INT_MAX;
        inorder(root,mine);
        return mine;

    }
};
```

Sum Root to Leaf Numbers

```
public:
void summ(TreeNode* root,int sum,int& ans){
    if(root==nullptr){
        return;
    }
    sum=(sum*10)+root->val;
    if(root->left==nullptr&&root->right==nullptr){
        ans+=sum;
        return;
    }
    summ(root->left,sum,ans);
    summ(root->right,sum,ans);
}
    int sumNumbers(TreeNode* root) {
        int ans=0;
        int sum=0;
        summ(root,sum,ans);
        return ans;

```

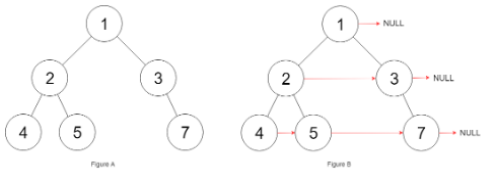
Invert Binary Tree

```
public:
    TreeNode* invertTree(TreeNode* root) {
        if(root==nullptr) return nullptr;
        queue<TreeNode*>q;
        q.push(root);
        while(!q.empty()){
                TreeNode* node=q.front();
                q.pop();
                swap(node->left,node->right);
            if(node->left){
                q.push(node->left);
            }
              if(node->right){
                q.push(node->right);
            }
        }
        return root;

    }
```

Lowest Common Ancestor of a Binary Tree

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if(root==nullptr||root==p||root==q) return root;
    TreeNode* left=lowestCommonAncestor(root->left,p,q);
    TreeNode* right=lowestCommonAncestor(root->right,p,q);
    if(left==nullptr) return right;
    if(right==nullptr) return left;
    return root;
}
```

Populating Next Right Pointers in Each Node II




```
Node* connect(Node* root) {
    if(root==nullptr) return nullptr;
   // vector<int>ans;
    queue<Node*>q;
    q.push(root);
    while(!q.empty()){
        int n=q.size();
        for(int i=0;i<n;i++){
            Node* node=q.front();
            q.pop();
            if(i<n-1){
                node->next=q.front();
            }
            if(node->left){
                q.push(node->left);
            }
            if(node->right){
                q.push(node->right);
            }
        }
    }
    return  root;
}
```

Morris Traversal ek special tree traversal technique hai jo **binary tree ko traverse karti hai bina recursion aur bina stack ke**, yani:

- Time Complexity = **O(N)**
- Extra Space = **O(1)**

Interview me bahut famous hai.

Flatten Binary Tree to Linked List

```
void preorder(TreeNode* root,vector<TreeNode*>& ans){
    if(root==nullptr) return ;
    ans.push_back(root);
    preorder(root->left,ans);
    preorder(root->right,ans);
}
    void flatten(TreeNode* root) {
       vector<TreeNode*>ans;
        preorder(root,ans);
    for(int i=1;i<ans.size();i++){
        ans[i-1]->left=nullptr;
        ans[i-1]->right=ans[i];
    }

    }
```